

GELİŞMİŞ RAG SİSTEMLERİ: AUTO-MERGING RETRIEVAL RAPORU

Hiyerarşik Düzüm Ayırıştırma (Hierarchical Node Parsing), Geri Çağırma Optimizasyonu ve Deneysel Karşılaştırma

1. GERİ ÇAĞIRMA PROBLEMINE GİRİŞ VE FRAGMENTASYON SIKINTISI

Geleneksel (temel / naive) Retrieval-Augmented Generation (RAG) sistemlerinde en yaygın yaklaşım, büyük belgeleri doğrudan küçük, bağımsız metin parçalarına (chunk) bölmektir. Bu parçalar vektör uzayında indekslenir ve kullanıcının sorusuyla en çok eşleşen parçalar doğrudan Büyük Dil Modeli'ne (LLM) beslenir. Ancak bu yöntemin ciddi bir yapısal problemi vardır: **Fragmentasyon (Parçalanma)**. Belgeleri küçük parçalara böldükçe, bilgi bütünlüğü ve bağlam kaybolur. Geri çağrılan metin parçaları aynı belgenin aynı bölümünden veya birbirine çok yakın paragraflardan gelse bile, LLM'in bağlam penceresine (context window) hangi sırada veya hangi biçimde gireceklerine dair hiçbir garanti yoktur. Bu kopukluk, LLM'in bağlamı doğru sentezlemesini zorlaştırır, halüsinasyon riskini artırır ve bilgi doğruluğunu olumsuz etkiler.

2. GÜNLÜK HAYATTAN BİR ANALOJİ: POST-IT NOTLARI VE KİTAP SAYFASI

Konuyu netleştirmek için günlük hayattan bir araştırma senaryosu düşünelim. Bir kütüphanede karmaşık bir konu hakkında araştırma yapıyorsunuz. Bir asistan size bu konuyla ilgili kitaptan kesilmiş 20 adet küçük, dağınık post-it notu getiriyor. Her notta sadece birer cümle yazıyor. Bu küçük notları okuyarak konunun bütünsel mantığını kavramaya çalışmak oldukça zordur, çünkü her cümlenin önündeki ve arkasındaki bağlamı göremezsiniz. Ancak asistanınız, bu küçük notlardan 4-5 tanesinin aslında aynı sayfaya veya aynı ana paragrafa ait olduğunu fark ettiğinde, size o dağınık küçük notları vermek yerine doğrudan **kitabın o tam sayfasını** açıp getirirse, konuyu çok daha hızlı ve doğru bir şekilde kavrayabilirsiniz.

Sistem Eşleşmesi:

- Yaprak Düzümler (Leaf Nodes / Post-it Notları):** Arama yapılan küçük, bağımsız metin parçacıklarıdır (örn. 128 token).
- Ebeveyn Düzümler (Parent Nodes / Kitap Sayfası):** Bu küçük parçacıkların içinde yer aldığı daha geniş, anlamlı metin bütünlüğüdür (örn. 512 veya 2048 token).
- Otomatik Birleştirme (Auto-merging):** Küçük parçalar geri çağırıldığında, eğer çoğunluğu aynı ebeveyne aitse, bunları LLM'e ayrı ayrı vermek yerine doğrudan ebeveyn düğüm ile değiştirerek bağlam bütünlüğünü koruma sürecidir.

3. AUTO-MERGING GERİ ÇAĞIRMA MEKANİZMASI VE ÇALIŞMA ADIMLARI

Auto-merging (Otomatik Birleştirme) retriever mekanizması, bir belgenin hiyerarşik bir ağaç yapısında ayrıştırılması esasına dayanır. Sistem şu aşamalardan geçerek çalışır:

- 1. Hiyerarşik Düğüm Ayrıştırma (Hierarchical Node Parsing):** Belgeler azalan boyutlarda hiyerarşik olarak bölünür. Örneğin, en üst katmanda 2048 tokenlik ana düğümler, onların altında 512 tokenlik orta katman düğümleri ve en altta 128 tokenlik yaprak (leaf) düğümler yer alır. Her yaprak düğüm, üstündeki ebeveyn düğüme ilişkisel bir referansla (parent link) bağlıdır.
- 2. Vektör Dizini (Vector Store Index):** Vektör veri tabanında arama yapabilmek amacıyla, **yalnızca en küçük yaprak düğümler (leaf nodes)** gömülür (embedded) ve dizine eklenir. Ara katmanlar ve ebeveyn düğümler ise bir belge deposunda (docstore) ilişkileriyle birlikte tutulur ancak doğrudan vektör aramasına tabi tutulmaz.
- 3. Birleştirme Mantiği ve Eşşik Değeri:** Kullanıcı sorgu gönderdiğinde sistem en alakalı yaprak düğümleri bulur (yüksek bir top-K değeri ile). Eğer bir ebeveyne ait yaprak düğümlerin belirli bir yüzdesi (threshold) geri çağırılmışsa, bu yaprak düğümler çıkarılır ve yerine doğrudan ebeveyn düğümün kendisi bağlama eklenir.
- 4. Yeniden Sıralama (Re-ranking):** Büyük düğümler bağlama eklendiğinde gereksiz token tüketimini önlemek için bir yeniden sıralama modeli (Reranker) uygulanır. Örneğin ilk aşamada çağrılan en alakalı 12 yaprak düğüm, birleştirmelerden sonra 10 düğüme düşebilir ve bir Reranker vasıtasıyla en kritik 6 düğüme indirilerek LLM'e sunulur.

4. PYTHON VE LLAMAINDEX İLE KOD UYGULAMASI

```
# LlamaIndex ile Auto-merging Retriever Kurulumu ve Yapılandırması

from llama_index.core.node_parser import HierarchicalNodeParser, get_leaf_nodes
from llama_index.core.storage.docstore import SimpleDocumentStore
from llama_index.core import StorageContext, VectorStoreIndex
from llama_index.core.retrievers import AutoMergingRetriever
from llama_index.core.query_engine import RetrieverQueryEngine
from llama_index.core.postprocessor import SentenceTransformerRerank

# 1. Hiyerarşik Düğüm Ayrıştırıcıyı Tanımlama (Faktör: 4 kat azalan boyutlar)
node_parser = HierarchicalNodeParser.from_defaults(
    chunk_sizes=[2048, 512, 128]
)
nodes = node_parser.get_nodes_from_documents(documents)
leaf_nodes = get_leaf_nodes(nodes)

# 2. Belge Deposunu (Docstore) Hazırlama ve Tüm Düğümleri Ekleme
docstore = SimpleDocumentStore()
docstore.add_documents(nodes)
storage_context = StorageContext.from_defaults(docstore=docstore)

# 3. Yalnızca Yaprak Düğümleri Vektör Dizinine Ekleme
vector_index = VectorStoreIndex(
    leaf_nodes,
    storage_context=storage_context,
    service_context=service_context
)

# 4. Auto-merging Retriever ve Reranker Modülünü Tanımlama
base_retriever = vector_index.as_retriever(similarity_top_k=12)
retriever = AutoMergingRetriever(
    base_retriever,
    vector_index.storage_context,
    simple_ratio_threshold=0.6 # %60 çocuk düğüm çağrıldığında birleştir
)
```

```
# Yeniden sıralama ile gereksiz token tüketimini önleme (Top 12 -> Top 6)
reranker = SentenceTransformerRerank(top_n=6, model="BAAI/bge-reranker-large")

# 5. Nihai Sorgu Motorunu (Query Engine) Oluşturma
query_engine = RetrieverQueryEngine.from_args(
    retriever,
    node_postprocessors=[reranker],
    service_context=service_context
)
```

5. RAG ÜÇLÜSÜ (RAG TRIAD) VE PERFORMANS DEĞERLENDİRMESİ

Gelişmiş RAG tekniklerinin başarısını doğrulamak için TruLens ve TruEra platformlarında **RAG Üçlüsü (RAG Triad)** metrik çerçevesi kullanılır. Bu üç metrik, sistemin dürüst, doğru ve alakalı yanıtlar üretmesini garanti altına alır:

- **Bağlam İlişkiseliliği (Context Relevance):** Geri çağrılan bağlaman kullanıcının sorusuyla ne kadar alakalı olduğunu ölçer.
- **Gerekçelendirme (Groundedness):** Üretilen yanıtın sadece geri çağrılan bağlama dayanıp dayanmadığını doğrular (halüsinasyon kontrolü).
- **Yanıt İlişkiseliliği (Answer Relevance):** LLM tarafından üretilen nihai yanıtın, sorulan soruya doğrudan cevap verip vermediğini ölçer.

6. DENEYSEL KARŞILAŞTIRMA: 2-KATMANLI VS. 3-KATMANLI MIMARI

TruEra üzerinde yapılan değerlendirmelerde, iki farklı hiyerarşik yapı (App 0 ve App 1) doğrudan karşılaştırılmıştır. Özellikle bütçeleme gibi detaylı kurumsal sorular içeren test senaryolarında elde edilen bulgular şunlardır:

Özellik / Metrik	2-Katmanlı Yapı (App 0)	3-Katmanlı Yapı (App 1)
Hiyerarşik Katman Boyutları	512 (Yaprak) -> 2048 (Ebeveyn)	128 (Yaprak) -> 512 (Ara) -> 2048 (Ebeveyn)
Çalışma Karmaşıklığı	Daha basit hiyerarşi, indeks oluşturması kolay	3 katmanlı ağaç, daha detaylı indeksleme
Bağlam İlişkiseliliği	Düşük (Kritik sorularda hata oranı yüksek)	Yaklaşık %20 Daha Yüksek (Doğru eşleşme)
Belge Doğruluğu (Groundedness)	Düşük (Kopuk bağlam nedeniyle halüsinasyon)	Önemli Ölçüde Yüksek (Bütünsel bağlam)
Token / API Maliyeti	Yüksek (Her yaprak düğüm en az 512 token)	Yaklaşık %50 Daha Ucuz (Yaprak düğümler 128 token)

7. ÖNEMLİ ÇIKARIMLAR VE MIMARI FARKLAR

- **Belge Türüne Göre Optimizasyon:** RAG sistemlerinde sabit bir ideal chunk boyutu yoktur. İş sözleşmeleri gibi uzun, hukuki belgeler ile fatura veya fatura benzeri yapılandırılmış/kısa belgeler, yapılarına en uygun farklı hiyerarşik katmanlar ve chunk boyutları ile optimize edilmelidir.
- **Auto-merging vs. Sentence Window Farkı:** Sentence windowing tekniği yalnızca ardışık (contiguous) cümleleri genişleterek bağlamı büyütür. Auto-merging ise hiyerarşik yapı sayesinde aynı ebeveyne ait olan ancak belgede peş peşe gelmeyen (non-contiguous) parçaları (örneğin paragrafın başı ile sonu) birleştirerek üstün bir bağlam oluşturma performansı sunar.